المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵓⵏⵉⵙⵉⵏⵉ - ⴰⵎⵜ ⵎⵓⵍⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Ecole nationale des sciences appliquées de Beni-Mellal

Niveau : 1ère année
Filière : IACS

TP 2 - Monolithe Modulaire

Réalisé par:
TOHANI Marwa

Encadré par:
Pr. ElBaghzaoui

1. Introduction

L'objectif de ce second TP était de faire évoluer le monolithe simple du TP1 vers une architecture modulaire. Les points clés étaient :

- Définir des frontières claires entre les domaines (Product, Customer, Order).
- Implémenter le pattern DTO (Data Transfer Object) pour isoler la base de données de l'API.
- Utiliser MapStruct pour automatiser les conversions Entity ⇔ DTO.
- Établir une communication entre services (le module Order dépend de Customer et Product).

2. Architecture du projet :

Nous avons appliqué une structure stricte pour chaque module :

- Modèles (Entities) : Représentent les tables PostgreSQL.
- DTOs : Objets utilisés pour les requêtes (Request) et les réponses (Response).
- Mappers : Interfaces MapStruct pour transformer les données sans code manuel répétitif.
- Services : Contiennent la logique métier (ex: vérification de l'existence d'un client).

3. Implementation des Modules

A. Module Customer (Indépendant)

Ce module gère les informations des clients. Nous avons ajouté une méthode cruciale : existsById(Long id). Elle permet aux autres modules de vérifier la validité d'un client sans accéder directement à la base de données des clients.

B. Module Order (Dépendant)

Le module Order est le plus complexe car il interagit avec les autres.

- Validation : Avant de créer une commande, le OrderService interroge le CustomerService.
- Historique : Un endpoint spécifique permet de récupérer toutes les commandes d'un client via son ID.

4. Code Source des Fonctionnalités Clés

- **Le Service de Commande (Logique de validation):**

```
L@Service
@RequiredArgsConstructor
public class OrderServiceImpl implements OrderService
{
   private final OrderRepository orderRepository;
   private final CustomerService customerService;
   private final OrderMapper orderMapper; // Injectez-le
ici

   @Override
   public boolean validateAndCreateOrder(Long
customerId, Double amount) {
```

```
if (customerService.existsById(customerId)) {
 Order newOrder = new Order();
 newOrder.setCustomerId(customerId);
 newOrder.setTotalAmount(amount); // Utilise le
paramètre reçu de Postman
 newOrder.setOrderDate(LocalDateTime.now());

 orderRepository.save(newOrder);
 return true;
 }
 return false;
 }
 @Override
 public List<OrderDTO> getOrdersByCustomerId(Long
customerId) {
 // 1. Get entities from database
 List<Order> orders =
orderRepository.findByCustomerId(customerId);

 // 2. Convert each Entity to a DTO using the mapper
 return orders.stream()
 .map(orderMapper::toDTO)
 .toList();
 }


}
```

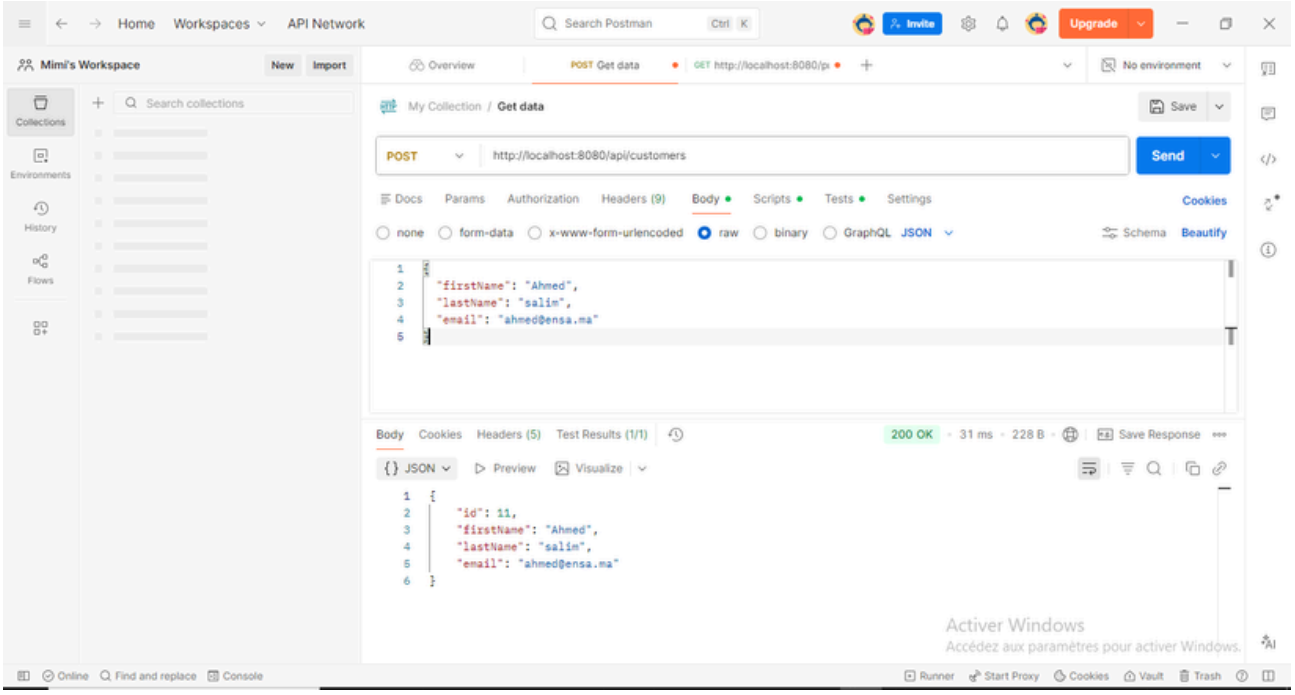
- **Le Mapper (order):**

```
ipackage com.ecommerce.monolith.order.mapper;

import com.ecommerce.monolith.order.dto.OrderDTO;
import com.ecommerce.monolith.order.model.Order;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

import java.util.List;

@Mapper(componentModel = "spring")
public interface OrderMapper {
   // Si l'entité et le DTO ont des noms différents, on
utilise @Mapping
   @Mapping(source = "customerId", target =
"customerId")
   @Mapping(source = "totalAmount", target =
"totalAmount")
   OrderDTO toDTO(Order order);
}
```

5. Tests et Validation (Postman))

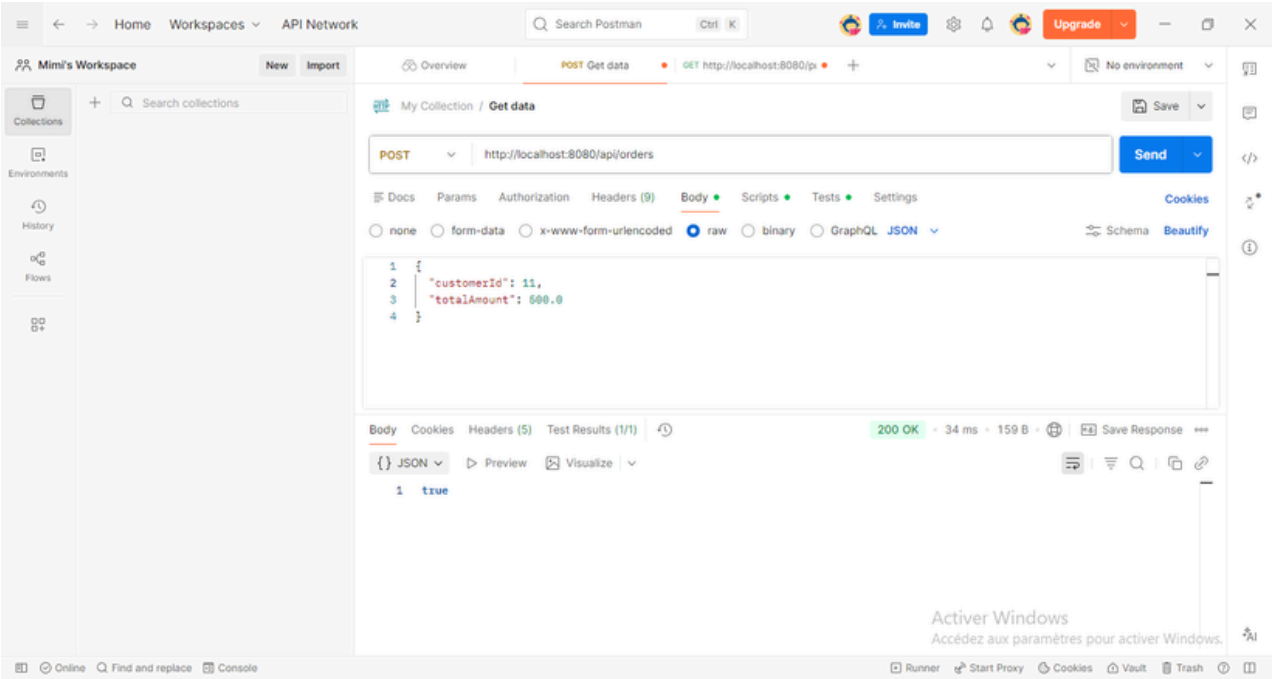
Étape 1 : Création du Client (Module Customer)

URL : http://localhost:8080/api/customers



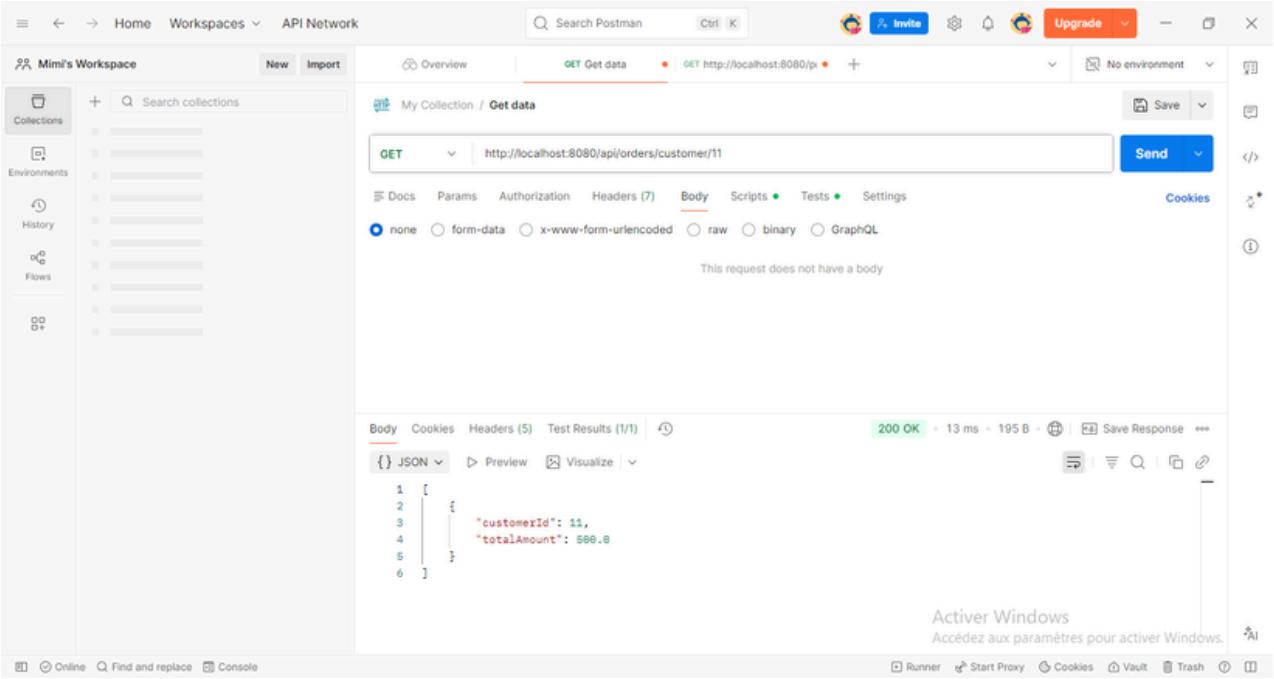
Étape 2 : Création de la Commande (Module Order)

URL : http://localhost:8080/api/orders



Étape 3 : Historique des commandes (Exigence 3)

URL : http://localhost:8080/api/orders/customer/11



4. Conclusion :

Ce TP a permis de transformer une application simple en un système robuste et évolutif. L'utilisation de MapStruct et des DTOs garantit que notre API est stable, même si la base de données change. Cette structure modulaire est la première étape indispensable avant de diviser ce monolithe en microservices.